

Software Testing Syllabus

Module 2

Functional Testing: Boundary value analysis, Robustness testing, Worst-case testing, Robust Worst testing for triangle problem, Nextdate problem and commission problem, Equivalence classes, Equivalence test cases for the triangle problem, NextDate function, and the commission problem, Guidelines and observations, Decision tables, Test cases for the triangle problem, NextDate function, and the commission problem, Guidelines and observations.

Fault Based Testing: Overview, Assumptions in fault based testing, Mutation analysis, Fault-based adequacy criteria, Variations on mutation analysis.

T1: Chapter 5, 6 & 7, T2: Chapter 16

Text Books:
1. Paul C. Jorgensen: Software Testing, A Craftsman's Approach, 3 rd Edition, Auerbach Publications, 2008. (Listed topics only from Chapters 1, 2, 5, 6, 7, 9, 10, 12, 13)
2. Mauro Pezze, Michal Young: Software Testing and Analysis – Process, Principles and Techniques, Wiley India, 2009. (Listed topics only from Chapters 3, 4, 16, 17, 20,21, 22,24)
3. Aditya P Mathur: Foundations of Software Testing, Pearson Education, 2008.(Listed topics only from Section 1.2 , 1.3, 1.4 ,1.5, 1.8,1.12,6. 2.1,6. 2.4)

Module-2

Functional Testing

Boundary Value Analysis

Q1.) Explain the Boundary Value Analysis.

Consider a function F of two variables x_1 and x_2 . When the function F is implemented as a program, the input variables x_1 and x_2 will have some (possibly unstated) boundaries:

$$a \leq x_1 \leq b$$

$$c \leq x_2 \leq d$$

The intervals $[a, b]$ and $[c, d]$ are referred to as the ranges of x_1 and x_2 . Strongly typed languages (such as Ada and Pascal) permit explicit definition of such variable ranges. Strong typing was to prevent programmers from making the kind of errors that result in faults that are easily revealed by boundary value testing.

Boundary value analysis focuses on the boundary of the input space to identify test cases. The rationale behind boundary value testing is that errors tend to occur near the extreme values of an input variable.

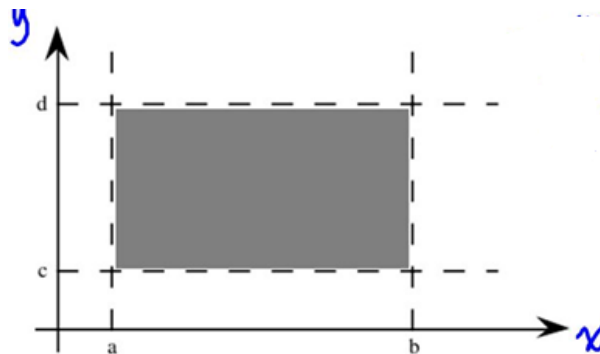


Figure 2.1: Input Domain of a Function of Two Variables

The basic idea of boundary value analysis is to use input variable values at their minimum, just above the minimum, a nominal value, just below their maximum, and at their maximum.

There is a commercially available testing tool (named T) that generates such test cases for a properly specified program. This tool has been successfully integrated with two popular front-end CASE tools (Teamwork from Cadre Systems, and Software Through Pictures from Interactive Development Environments).

The T tool refers to these values as min, min+, nom, max- and max;

- Min ----- - Minimal
- Min+ ----- - Just above Minimal
- Nom ----- - Average

- Max- ----- - Just below Maximum
- Max ----- - Maximum

The next part of boundary value analysis is based on a critical assumption; it's known as the "single fault" assumption in reliability theory. This says that failures are only rarely the result of the simultaneous occurrence of two (or more) faults. Thus, the boundary value analysis test cases are obtained by holding the values of all but one variable at their nominal values, and letting that variable assume its extreme values.

The boundary value analysis test cases for our function F of two variables are:

$\{ \langle x1nom, x2min \rangle, \langle x1nom, x2min+ \rangle, \langle x1nom, x2nom \rangle, \langle x1nom, x2max- \rangle, \langle x1nom, x2max \rangle, \langle x1min, x2nom \rangle, \langle x1min+, x2nom \rangle, \langle x1nom, x2nom \rangle, \langle x1max-, x2nom \rangle, \langle x1max, x2nom \rangle \}$

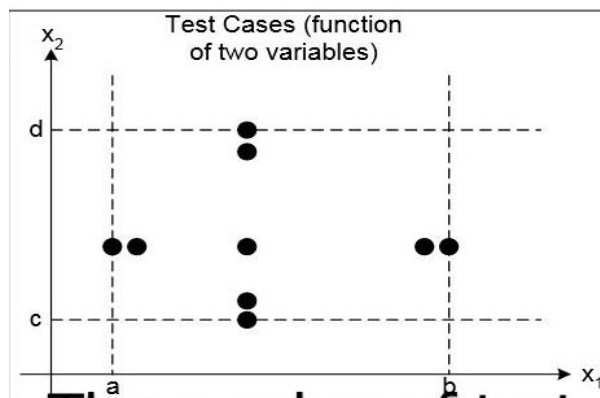


Figure 2.2: Function of two variables

Generalizing Boundary Value Analysis

The basic boundary value analysis technique can be generalized in two ways:

- by the number of variables, and
- by the kinds of ranges.

Generalizing the number of variables is easy: if we have a function of n variables, hold all but one at the nominal values, and let the remaining variable assume the min, min+, nom, max- and max values, and repeat this for each variable. Thus, for a function of n variables, boundary value yields $4n + 1$ test cases.

Generalizing ranges depends on the nature (or more precisely, the type) of the variables themselves.

In the NextDate function, for example, we have variables for the month, the day, and the year. In a FORTRAN-like language, we would most likely encode these, so that January would correspond to 1, February to 2, and so on. In a language that supports user defined types (like Pascal), we could define the variable month as an enumerated type {Jan., Feb., . . . , Dec.}. Either way, the values for min, min+, nom, max- and max are clear from the context. When a variable has discrete, bounded values, as the variables in the commission problem have, the min, min+, nom, max- and max are also easily determined.

Limitations of Boundary Value Analysis

Q 2.) What are the advantages and limitations of Boundary value analysis?

Q) What are the limitations of boundary value analysis.

Q) Explain boundary value analysis. Mention its limitations. Derive BVA test cases for the triangle problem

- **Advantages of Boundary Value Analysis**

1. It is easier and faster to find defects using this technique. This is because the density of defects at boundaries is more.
2. Instead of testing will all set of test data, we only pick the one at the boundaries. So, the overall test execution time reduces.

- **Limitations of BVA**

1. Boundary Value Analysis works well when the Program Under Test (PUT) is a “function of several independent variables that represent bounded physical quantities”. When these conditions are met BVA works well but when they are not, we can find deficiencies in the results.
2. For example, the NextDate problem, where Boundary Value Analysis would place an even testing regime equally over the range, tester’s intuition and common sense shows that we require more emphasis towards the end of February or on leap years.
3. The reason for this poor performance is that BVA cannot compensate or take into consideration the nature of a function or the dependencies between its variables. This lack of intuition or understanding for the variable nature means that BVA can be seen as quite rudimentary.

Robustness Testing

Q 3.) Explain boundary value analysis and BVA robust in detail with function of two variables and show how to prepare test input sets.

Robustness testing can be seen as an extension of Boundary Value Analysis. The idea behind Robustness testing is to test for clean and dirty test cases. By clean I mean input variables that lie in the legitimate input range. By dirty I mean using input variables that fall just outside this input domain.

In addition to the aforementioned 5 testing values (min, min+, nom, max-, max) we use two more values for each variable (min-, max+), which are designed to fall just outside of the input range.

If we adapt our function f to apply to Robustness testing, we find the following equation:

$$f = 6n + 1$$

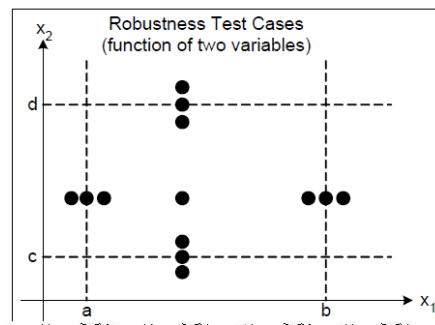


Figure 2.3: Robustness testing

Worst Case Testing

Boundary Value analysis uses the critical fault assumption and therefore only tests for a single variable at a time assuming its extreme values. By disregarding this assumption, we are able to test the outcome if more than one variable were to assume its extreme value. In an electronic circuit this is called Worst Case Analysis. In Worst-Case testing we use this idea to create test cases.

To generate test cases, we take the original 5-tuple set (min, min+, nom, max-, max) and perform the Cartesian product of these values. The end product is a much larger set of results than we have seen before.

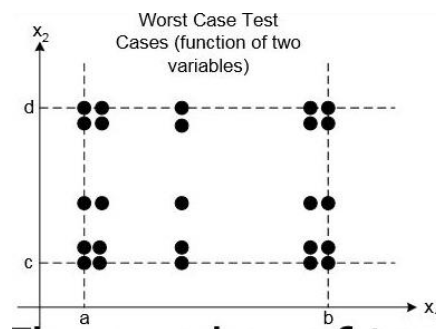


Figure 2.4: Worst Case Testing

Special Value Testing

Special value testing occurs when a tester uses his/her domain knowledge, experience with similar programs, and information about “soft spots” to devise test cases.

This testing is also called as “ad hoc testing” or “seat of the pants/skirt” testing.

Robust Worst-Case Testing

If the function under test were to be of the greatest importance, we could use a method named Robust Worst-Case testing which as the name suggests draws its attributes from Robust and Worst-Case testing.

Test cases are constructed by taking the Cartesian product of the 7-tuple set defined in the Robustness testing. Obviously, this results in more effort required to produce largest set of test results.

We can see that the function f (to calculate the number of test cases required) can be adapted to calculate the amount of Robust Worst-Case test cases. As there are now 7 values each variable can assume, we find the function f to be:

$$f = 7^n$$

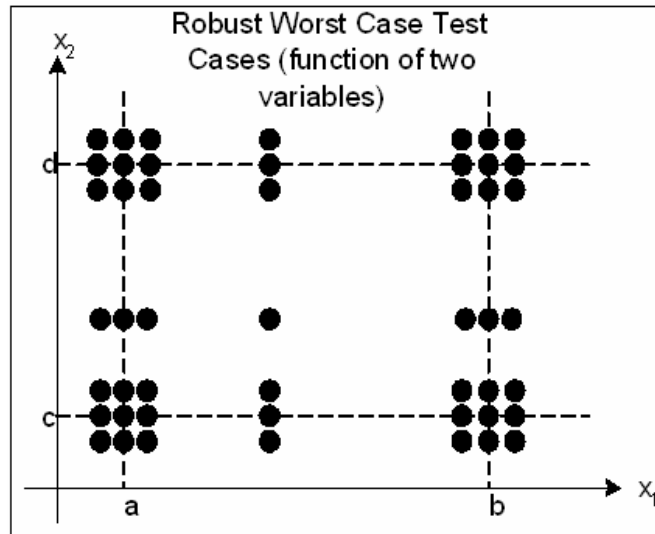


Figure 2.5: Robust Worst-Case Testing

Example:

Q 4.) Derive test cases for the triangle problem.

Each of the three continuing examples is a function of three variables. Printing all the test cases from all the methods for problem is very space consuming, so we'll just have selected examples.

Table 2.1: Test Cases for the Triangle Problem

TC ID	Input			Expected Output	Status
	a	b	c		
01	100	100	1	The triangle is Isosceles	Pass
02	100	100	2	The triangle is Isosceles	Pass
03	100	100	100	The triangle is Equilateral	Pass
04	100	100	199	The triangle is Isosceles	Pass
05	100	100	200	The triangle construction is not possible	Pass
06	100	1	100	The triangle is Isosceles	Pass
07	100	2	100	The triangle is Isosceles	Pass
08	100	199	100	The triangle is Isosceles	Pass
09	100	200	100	The triangle construction is not possible	Pass
10	1	100	100	The triangle is Isosceles	Pass
11	2	100	100	The triangle is Isosceles	Pass
12	199	100	100	The triangle is Isosceles	Pass
13	200	100	100	The triangle construction is not possible	Pass

Table 2.2: Sample Test Cases for the worst-case Triangle Problem

TC ID	Input			Expected Output	Status
	a	b	c		
1.	1	1	1	The triangle is Equilateral	Pass
2.	1	1	2	The triangle construction is not possible	Pass
3.	1	1	5	The triangle construction is not possible	Pass
4.	1	1	9	The triangle construction is not possible	Pass
5.	1	1	10	The triangle construction is not possible	Pass
6.	1	2	1	The triangle construction is not possible	Pass
7.	1	2	2	The triangle is Isosceles	Pass
8.	1	2	5	The triangle construction is not possible	Pass
9.	1	2	9	The triangle construction is not possible	Pass
10.	1	2	10	The triangle construction is not possible	Pass
11.	1	5	1	The triangle construction is not possible	Pass
12.	1	5	2	The triangle construction is not possible	Pass
13.	1	5	5	The triangle is Isosceles	Pass
14.	1	5	9	The triangle construction is not possible	Pass
15.	1	5	10	The triangle construction is not possible	Pass
16.	1	9	1	The triangle construction is not possible	Pass
17.	1	9	2	The triangle construction is not possible	Pass
18.	1	9	5	The triangle construction is not possible	Pass
19.	1	9	9	The triangle is Isosceles	Pass
20.	1	9	10	The triangle construction is not possible	Pass
21.	1	10	1	The triangle construction is not possible	Pass
22.	1	10	2	The triangle construction is not possible	Pass
23.	1	10	5	The triangle construction is not possible	Pass
24.	1	10	9	The triangle construction is not possible	Pass
25.	1	10	10	The triangle is Isosceles	Pass

Test Cases for the Commission Problem

Rather than go through 125 boring test cases again, we'll look at some more interesting test cases for the commission problem. This time we'll look at boundary values for the output range, especially near the threshold points of \$1000 and \$1800.

The volume below the lower plane corresponds to sales below the \$1000 threshold. The volume between the two planes is the 15% commission range. Part of the reason for using the output range to determine test cases is that cases from the input range are almost all in the 20% zone. We want to find input variable combinations that stress the boundary values: \$100, \$1000, \$1800, and \$7800. These test cases were developed with a spreadsheet, which saves a

lot of calculator pecking. The minimum and maximum were easy, and the numbers happen to work out so that the border points are easy to generate.

Table 2.3: Test Cases for the Commission Problem

TC ID	Input			Expected Output		Actual Output (Commission)	Status
	Locks	Stocks	Barrels	Sales	Commission		
01	35	40	1	2800	420	420	Pass
02	35	40	2	2825	425	425	Pass
03	35	40	45	3900	640	640	Pass
04	35	40	89	5000	860	860	Pass
05	35	40	90	5025	865	865	Pass
06	35	1	45	2730	406	406	Pass
07	35	2	45	2760	412	412	Pass
08	35	79	45	5070	874	874	Pass
09	35	80	45	5100	880	880	Pass
10	1	40	45	2370	334	334	Pass
11	2	40	45	2415	343	343	Pass
12	69	40	45	5430	940	940	Pass
13	70	40	45	5475	955	955	Pass

Next Date problem

Table 2.4: Test Cases for the Next Date Problem

TC ID	Input			Expected Output	Status
	Day	Month	Year		
01	1	6	1912	2-6-1912	Pass
02	2	6	1912	3-6-1912	Pass
03	15	6	1912	16-6-1912	Pass
04	30	6	1912	1-7-1912	Pass
05	31	6	1912	Invalid day	Pass
06	15	1	1912	16-1-1912	Pass
07	15	2	1912	16-2-1912	Pass
08	15	11	1912	16-11-1912	Pass
09	15	12	1912	16-12-1912	Pass
10	15	6	1812	16-6-1812	Pass
11	15	6	1813	16-6-1813	Pass
12	15	6	2011	16-6-2011	Pass
13	15	6	2012	16-6-2012	Pass

Equivalence Classes

Equivalence classes form a partition of a set, where partition refers to a collection of mutually disjoint subsets whose union is the entire set.

This has two important implications for testing:

- The fact that the entire set is represented provides a form of completeness, and
- The disjointness assures a form of non-redundancy.

The idea of equivalence class testing is to identify test cases by using one element from each equivalence class. If the equivalence classes are chosen wisely, this greatly reduces the potential redundancy among test cases.

The key (and the craft!) of equivalence class testing is the choice of the equivalence relation that determines the classes.

Types or Variants of Equivalence class testing

Q 5.) Briefly explain the variants of Equivalence Class Testing.

Q) Explain weak normal, weak robust and strong robust equivalence class testing with next date problem as an example

Q) Explain different types of equivalence class testing in detail

1. Weak Normal Equivalence Class Testing
2. Strong Normal Equivalence Class Testing
3. Weak Robust Equivalence Class Testing
4. Strong Robust Equivalence Class Testing

1. Weak Equivalence Class Testing

The word 'weak' means 'single fault assumption'. This type of testing is accomplished by using one variable from each equivalence class in a test case.

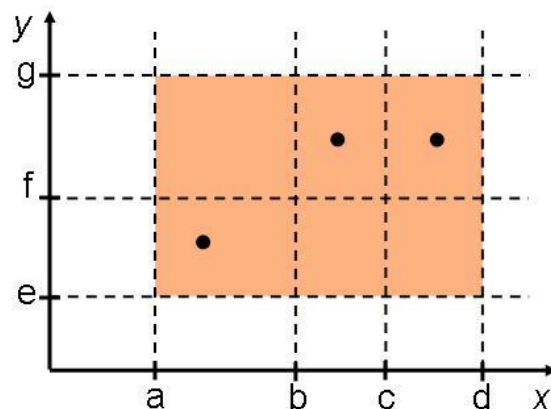


Figure 2.5: Weak Normal Equivalence Class Testing

Table 2.5: Weak Normal Equivalence class Test Cases for the Triangle Problem

TC ID	Test Case Description	Input			Expected Output	Actual Output	Status
		a	b	c			
1	WN1	5	5	5	The triangle is Equilateral	The triangle is Equilateral	Pass
2	WN2	2	2	3	The triangle is isosceles	The triangle is isosceles	Pass
3	WN3	3	4	5	The triangle is Scalene	The triangle is Scalene	Pass
4	WN4	4	1	2	The triangle construction is not possible	The triangle construction is not possible	Pass

This set of test cases uses one value from each equivalence class. We identify these in a systematic way, hence the apparent pattern.

2. Strong Normal Equivalence Class Testing

Strong equivalence class testing is based on the Cartesian Product of the partition subsets. The Cartesian Product guarantees that we have a notion of “completeness” in two senses: we cover all the equivalence classes, and we have one of each possible combination of inputs.

This type of testing is based on the multiple fault assumption theory. So, now we need test cases from each element of the Cartesian product of the equivalence classes.

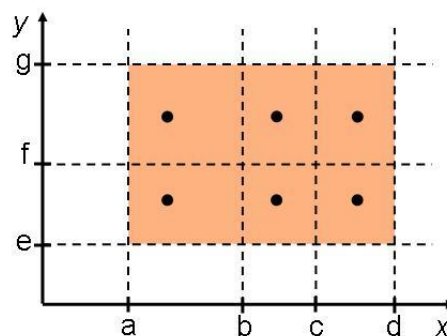


Figure 2.6: Strong Normal Equivalence Class Testing

3. Weak Robust Equivalence Class Testing

The word ‘weak’ means single fault assumption theory and the word ‘Robust’ refers to invalid values.

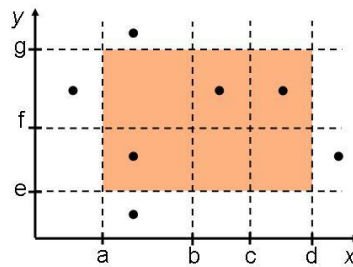


Figure 2.7: Weak Robust Equivalence Class Testing

4. Strong Robust Equivalence Class Testing

‘Robust’ means consideration of invalid values and the ‘strong’ means multiple fault assumption. We obtain the test cases from each element of the Cartesian product of all the equivalence classes.

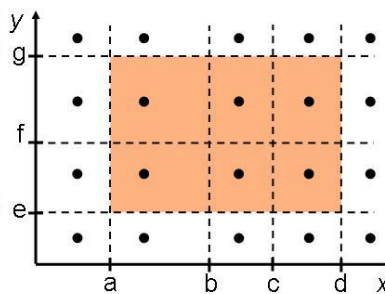


Figure 2.8: Strong Robust Equivalence Class Testing

Traditional Equivalence Class Testing

The traditional view of equivalence class testing defines equivalence classes in terms of validity. For each input variable, there are valid and invalid values; in the traditional approach, these are identified and numbered, and then incorporated test cases in the weak sense.

As an example, consider the valid ranges defined for the variables in the Commission problem:

$$1 \leq \text{lock} \leq 70$$

$$1 \leq \text{stock} \leq 80$$

$$1 \leq \text{barrel} \leq 90$$

Invalid equivalence classes

Lock < 1	Lock > 70
Stock < 1	Stock > 80
Barrel < 1	Barrel > 90

Given these valid and invalid sets of inputs, the traditional equivalence testing strategy identifies test cases as follows:

1. For valid inputs, use one value from each valid class -weak equivalence class testing. (Note that each input in these test cases will be valid.)

2. For invalid inputs, a test case will have one invalid value and the remaining values will all be valid. (Thus a “single failure” should cause the test case to fail.)

There are two problems with traditional equivalence testing.

1. The specification does not define what the expected output for an invalid test case should be. Thus, testers spend a lot of time defining expected outputs for these cases.
2. Strongly typed languages eliminate the need for the consideration of invalid inputs.

Table 2.6: Output based Triangle problem Equivalence class test cases

TC ID	Test Case Description	Input			Expected Output	Status
		a	b	c		
1	OE1	5	5	5	The triangle is Equilateral	Pass
2	OE2	2	2	3	The triangle is isosceles	Pass
3	OE3	3	4	5	The triangle is Scalene	Pass
4	OE4	4	1	2	The triangle construction is not possible	Pass

If we base equivalence classes on the input domain, we obtain a richer set of test cases. What are some of the possibilities for the three integers, a, b, and c?

- They can all be equal,
- Exactly one pair can be equal (this can happen three ways), or
- None can be equal.

$D1 = \{ \langle a, b, c \rangle : a = b = c \}$

$D2 = \{ \langle a, b, c \rangle : a = b, a \neq c \}$

$D3 = \{ \langle a, b, c \rangle : a = c, a \neq b \}$

$D4 = \{ \langle a, b, c \rangle : b = c, a \neq b \}$

$D5 = \{ \langle a, b, c \rangle : a \neq b, a \neq c, b \neq c \}$

As a separate question, we can apply the triangle property to see if they even constitute a triangle.

$D6 = \{ \langle a, b, c \rangle : a \geq (b + c) \}$

$D7 = \{ \langle a, b, c \rangle : b \geq (a + c) \}$

$D8 = \{ \langle a, b, c \rangle : c \geq (a + b) \}$

Equivalence Class Test Cases for the NextDate Function

Q 6.) Derive equivalence class test cases for next date problem

The NextDate Function illustrates very well the craft of choosing the underlying equivalence relation. It is also a good example on which to compare traditional, weak, and strong forms of

equivalence class testing. NextDate is a function of three variables, month, day, and year, and these have ranges defined as follows:

$$1 \leq \text{month} \leq 12$$

$$1 \leq \text{day} \leq 31$$

$$1812 \leq \text{year} \leq 2012$$

Traditional Test Cases

The valid equivalence classes are

$$M1 = \{\text{month} : 1 \leq \text{month} \leq 12\}$$

$$D1 = \{\text{day} : 1 \leq \text{day} \leq 31\}$$

$$Y1 = \{\text{year} : 1812 \leq \text{year} \leq 2012\}$$

The invalid equivalence classes are

$$M2 = \{\text{month} : \text{month} < 1\}$$

$$M3 = \{\text{month} : \text{month} > 12\}$$

$$D2 = \{\text{day} : \text{day} < 1\}$$

$$D3 = \{\text{day} : \text{day} > 31\}$$

$$Y2 = \{\text{year} : \text{year} < 1812\}$$

$$Y3 = \{\text{year} : \text{year} > 2012\}$$

Table 2.7: Output based NextDate function Equivalence class test cases

TC Id	Test Case Description	Input Data			Expected Output	Actual Output	Status
		DD	MM	YYYY			
1	Testing for Valid input changing the day within the month.	15	6	1900	16-6-1900	16-6-1900	pass

If we more carefully choose the equivalence relation, the resulting equivalence classes will be more useful.

What must be done to an input date?

- If it is not the last day of a month, the NextDate function will simply increment the day value.
- At the end of a month, the next day is 1 and the month is incremented.
- At the end of a year, both the day and the month are reset to 1, and the year is incremented.
- Finally, the problem of leap year makes determining the last day of a month interesting.

With all this in mind, we might postulate the following equivalence classes:

$$M1 = \{\text{month} : \text{month has 30 days}\}$$

$$M2 = \{\text{month} : \text{month has 31 days}\}$$

$$M3 = \{\text{month} : \text{month is February}\}$$

$$D1 = \{\text{day} : 1 < \text{day} \leq 28\}$$

D2 = {day: day=29}

D3 = {day: day=30}

D4 = {day: day=31}

Y1 = {year: year=1900}

Y2 = {year: $1812 \leq \text{year} \leq 2012$ AND (year $\neq 1900$) AND (year $\equiv 0 \pmod{4}$)}

Y3 = {year: ($1812 \leq \text{year} \leq 2012$ AND year $\not\equiv 0 \pmod{4}$)}

By choosing separate classes for 30- and 31-day months, we simplify the last day of the month question.

By taking February as a separate class, we can give more attention to leap year questions.

We also give special attention to day values: days in D₁ are (nearly) always incremented, while days in D₄ only have meaning for months in M₂.

Finally, we have three classes of years, the special case of the year 1900, leap years, and common years. This isn't a perfect set of equivalence classes, but its use will reveal many potential errors.

Weak Equivalence Class Test Cases

Table 2.8: Weak Robust NextDate function Equivalence class test cases

TC Id	Input Data			Expected Output	Status
	DD	MM	YYYY		
WR1	15	6	1900	16-6-1900	Pass
WR2	-1	6	1900	Invalid day	Pass
WR3	32	6	1900	Invalid day	Pass
WR4	15	-1	1900	Invalid month	Pass
WR5	13	13	1900	Invalid month	Pass
WR6	6	15	1802	Invalid year	Pass
WR7	6	15	2013	Invalid year	Pass

Strong Equivalence Class Test Cases

Using the same equivalence classes, we find the strong equivalence class test cases shown in the table below.

Table 2.9: Strong Robust NextDate function Equivalence class test cases

TC ID	Input Data			Expected Output	Status
	DD	MM	YYYY		
SR1	15	-1	1900	Invalid month	Pass
SR2	-1	6	1900	Invalid day	Pass
SR3	6	15	1809	Invalid Year	Pass
SR4	-1	-1	1900	Invalid day Invalid month	Pass

SR5	-1	6	1811	Invalid day Invalid year	Pass
SR6	6	-1	1811	Invalid month Invalid year	Pass
SR7	-1	-1	1811	Invalid day Invalid month Invalid year	Pass

Equivalence Class Test Cases for the Commission Problem

The Input domain of the Commission Problem is “naturally” partitioned by the limits on locks, stocks, and barrels. These equivalence classes are exactly those that would also be identified by traditional equivalence class testing. The first class is the valid input, the other two are invalid. The input domain equivalence classes lead to very unsatisfactory sets of test cases.

Lock L1 = {lock: $1 \leq \text{lock} \leq 70$ }

L2 = {lock: lock < 1}

L3 = {lock: lock > 70}

Stock S1 = {stock: $1 \leq \text{stock} \leq 80$ }

S2 = {stock: stock < 1}

S3 = {stock: stock > 80}

Barrel B1 = {barrel: $1 \leq \text{barrel} \leq 90$ }

B2 = {barrel: barrel < 1}

B3 = {barrel: barrel > 90}

Weak Input Domain Equivalence Class Test Cases

These classes yield the following weak equivalence class test cases. As with the other examples, the inputs are mechanically selected.

Table 2.10: Weak & Strong Normal Equivalence class Test Cases:

TC Id	Input Data			Sales	Expected Output (Commission)	Status
	Locks	Stocks	Barrels			
WN1, SN1	35	40	45	640	3900	Pass

Table 2.11: Weak Robust Equivalence class Test Cases:

TC Id	Input Data			Sales	Expected Output (Commission)	Result
	Locks	Stocks	Barrels			
WR1	10	10	10	\$100	10	Pass
WR2	-1	40	45		Program terminates	Pass

WR3	-2	40	45	-----	Value of the locks is out of range	Pass
WR4	71	40	45	-----	Value of the locks is out of range	Pass
WR5	35	-1	45	-----	Value of the stocks is out of range	Pass
WR6	35	81	45	-----	Value of the stocks is out of range	Pass
WR7	35	40	-1	-----	Value of the barrels is out of range	Pass
WR8	35	40	91	-----	Value of the barrels is out of range	Pass

Table 2.12: Strong Robust Equivalence class Test Cases:

TC Id	Input Data			Sales	Expected Output (Commission)	Status
	Locks	Stocks	Barrels			
SR1	-2	40	45	-----	Value of the locks is out of range	Pass
SR2	35	-1	45	-----	Value of the stocks is out of range	Pass
SR3	35	40	-2	-----	Value of the barrels is out of range	Pass
SR4	-2	-1	45	-----	Values of locks the locks is out of range Values of stocks is out of range	Pass
SR5	-2	40	-1	-----	Values of locks the locks is out of range Value of the barrels is out of range	Pass
SR6	35	-1	-1	-----	Values of locks the stocks is out of range Value of the barrels is out of range	Pass
SR7	-2	-1	-1	-----	Values of the locks is out of range Value of the stocks is out of range Value of the barrels is out of range	Pass

Output Range Equivalence Class Test Cases

Notice that, of 27 test cases, only one is a legitimate input. If we were really worried about error cases, this might be a good set of test cases. It can hardly give us a sense of confidence about the calculation portion of the problem, however. We can get some help by considering equivalence classes defined on the output range. Sales is a total amount based on the number of locks, stocks and barrels sold:

Sales = 45 X locks + 30 X stocks + 25 X barrels

L1 = {lock, stock, barrel: sales < 1000}

$L2 = \{\text{lock, stock, barrel: } 1000 \leq \text{sales} \leq 1800\}$

$L2 = \{\text{lock, stock, barrel: sales} > 1800\}$

Elements of L1 are points with integer coordinates in the pyramid near the origin. Elements of L2 are points in the “triangular slice” between the pyramid and the rest of the input space.

Table 2.12: Output based Equivalence class Test Cases:

TC Id	Input Data			Sales	Expected Output (Commission)	Actual Output	Status
	Locks	Stocks	Barrels				
1	5	5	5	500	a1:50	500	Pass
2	15	15	15	1500	a2: 175	1500	Pass
3	25	25	25	2500	a3: 360	2500	Pass

Guidelines and Observations

Now that we have gone through three examples, we conclude with some observations about, and guidelines for equivalence class testing.

1. The traditional form of equivalence class testing is generally not as thorough as weak equivalence class testing, which in turn, is not as thorough as the strong form of equivalence class testing.
2. The only time it makes sense to use the traditional approach is when the implementation language is not strongly typed.
3. If error conditions are a high priority, we could extend strong equivalence class testing to include invalid classes.
4. Equivalence class testing is appropriate when input data is defined in terms of ranges and sets of discrete values. This is certainly the case when system malfunctions can occur for out-of-limit variable values.
5. Equivalence class testing is strengthened by a hybrid approach with boundary value testing. (We can “reuse” the effort made in defining the equivalence classes.)
6. Equivalence class testing is indicated when the program function is complex. In such cases, the complexity of the function can help identify useful equivalence classes, as in the NextDate function.
7. Strong equivalence class testing makes a presumption that the variables are independent when the Cartesian Product is taken. If there are any dependencies, these will often generate “error” test cases, as they did in the NextDate function.

Decision Table Based Testing

Decision Tables Technique

Q 7.) Write a short note on decision table with an example

To identify test cases with decision tables, we interpret conditions as inputs, and actions as outputs. Sometimes, conditions end up referring to equivalence classes of inputs, and actions refer to major functional processing portions of the item being tested. The rules are then interpreted as test cases.

Use of Don't Care entries has a subtle effect on the way in which complete decision tables are recognized. For limited entry decision tables, if there are n conditions, there must be 2^n rules. When Don't Care entries really indicate that the condition is irrelevant, we can develop a rule count as follows. Rules in which no Don't Care entries occur count as one rule. Each Don't Care entry in a rule doubles the count of that rule.

Q 8.) Explain the format of decision table. Build decision table for simple version of triangle problem.**Table 2.13: Decision table for Triangle problem**

Condition & Actions	Rules										
	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10	R11
c1: $a < b + c$?	F	T	T	T	T	T	T	T	T	T	T
c2: $b < a + c$?	-	F	T	T	T	T	T	T	T	T	T
c3: $c < a + b$?	-	-	F	T	T	T	T	T	T	T	T
c4: $a = b$?	-	-	-	T	T	T	T	F	F	F	F
c5: $a = c$?	-	-	-	T	T	F	F	T	T	F	F
c6: $b = c$?	-	-	-	T	F	T	F	T	F	T	F
Rule Count	32	16	8	1	1	1	1	1	1	1	1
a1: Not a Triangle	X	X	X								
a2: Scalene											X
a3: Isosceles							X		X	X	
a4: Equilateral				X							
a5: Impossible					X	X		X			

Since we should only have eight rules (3 three impossible cases), we clearly have a problem. To see where the problem lies, we expand each of the three rules, replacing the — entries with the T and F possibilities.

Test Cases for the Triangle Problem

We obtain eleven functional test cases: three impossible cases, three ways to fail the triangle property, one way to get an equilateral triangle, one way to get a scalene triangle, and three ways to get an isosceles triangle. If we extended the decision table to show both ways to fail an inequality, we would pick up three more test cases (where one side is exactly the sum of

the other two). Some judgment is required in this because of the exponential growth of rules. In this case, we would end up with many more don't care entries, and more impossible rules.

Table 2.14: Decision table Test cases for Triangle problem

TC ID	Input			Expected Output	Observation
	a	b	c		
01	7	2	3	The triangle construction is not possible	Pass
02	2	7	3	The triangle construction is not possible	Pass
03	3	2	7	The triangle construction is not possible	Pass
04	8	8	8	The triangle is Equilateral	Pass
05	?	?	?	Impossible	Pass
06	?	?	?	Impossible	Pass
07	6	6	7	The triangle is Isosceles	Pass
08	?	?	?	Impossible	Pass
09	6	7	6	The triangle is Isosceles	Pass
10	7	6	6	The triangle is Isosceles	Pass
11	3	4	5	The triangle is Scalene	Pass

Test Cases for the NextDate Function

The NextDate function was chosen because it illustrates the problem of dependencies in the input domain. This makes it a perfect example for decision table-based testing, because decision tables can highlight the dependencies.

First Try

Identifying appropriate conditions and actions presents an opportunity for craftsmanship.

M1 = {month: month has 30 days}

M2 = {month: month has 31 days}

M3 = {month: month is February}

D1 = {day: $1 < \text{day} \leq 28$ }

D2 = {day: day=29}

D3 = {day: day=30}

D4 = {day: day=31}

Y1 = {year: year is a leap year}

Y2 = {year: year is a common year}

Table 2.15: First Try Decision table Test cases for NextDate function

conditions												
c1: month in M1?	T											
c2: month in M2?		T										
c3: month in M3?			T									
c4: day in D1?												
c5: day in D2?												
c6: day in D3?												
c7: day in D4?												
c8: year in Y1?												
a1: impossible												
a2: next date												

This decision table will have 256 rules, many of which will be impossible. If we wanted to show why these rules were impossible, we might revise our actions to the following: Actions

a1: Too many days in a month

a2: Cannot happen in a common year

a3: Compute the next date

Second Try

To illustrate another decision table technique, this time we'll develop an Extended Entry Decision Table, and we'll take a closer look at the action stub. In making an Extended Entry Decision Table, we must ensure that the equivalence classes form a true partition of the input domain. If there were any "overlap" among the rule entries, we would have a redundant case in which more than one rule could be satisfied. Here, Y2 is the set of years between 1812 and 2012 evenly divisible by four excluding the year 1900.

M1 = {month: month has 30 days}

M2 = {month: month has 31 days}

M3 = {month: month is February}

D1 = {day: $1 < \text{day} \leq 28$ }

D2 = {day: day=29}

D3 = {day: day=30}

D4 = {day: day=31}

Y1 = {year: year=1900}

Y2 = {year: 1812≤year≤2012 AND (year≠1900) AND (year=0 mod 4)}

Y3 = {year: (1812≤year≤2012 AND year ≠0 mod 4)}

Table 2.16: Second Try Decision table Test cases for NextDate function

Conditions	1	2	3	4	5	6	7	8
c1: month in	M1	M1	M1	M1	M2	M2	M2	M2
c2: day in	D1	D2	D3	D4	D1	D2	D3	D4
c3: year in	-	-	-	-	-	-	-	-
Rule Count	3	3	3	3	3	3	3	3
actions								
a1: impossible					X			
a2: increment day	X	X			X	X	X	
a3: reset day			X					X
a4: increment month			X					?
a5: reset month								?
a6: increment year								?
conditions	9	10	11	12	13	14	15	16
c1: month in	M3	M3	M3	M3	M3	M3	M3	M3
c2: day in	D1	D1	D1	D2	D2	D2	D3	D3
c3: year in	Y1	Y2	Y3	Y1	Y2	Y3	-	-
Rule Count	1	1	1	1	1	1	3	3
actions								
a1: impossible				X		X	X	X
a2: increment day		X						
a3: reset day	X		X		X			

a4: increment month	X		X		X			
a5: reset month								
a6: increment year								

This decision table has 36 rules, and corresponds to the Cartesian Product of the equivalence classes. We still have the problem with logically impossible rules, but this formulation helps us identify the expected outputs of a test case. If you develop this table, we are able to find some cumbersome problems with December (in rule 8). We fix these next.

Third Try

Revised NextDate Domain Equivalence Classes by considering end of year.

M1 = {month: month has 30 days}

M2 = {month: month has 31 days}

M3 = {month: month is February}

D1 = {day: $1 < \text{day} \leq 28$ }

D2 = {day: day=29}

D3 = {day: day=30}

D4 = {day: day=31}

Y1 = {year: year is a leap year}

Y2 = {year: year is a common year}

The Cartesian product of these contains 40 elements. The full decision table is it has 22 rules, compared to the 36 of the second try. Here we have a 22 rule decision table that gives a clearer picture of the NextDate function than does the 36 rule decision table. The first five rules deal with 30 day months; notice that the leap year considerations are irrelevant. The next two sets of rules (6 - 10 and 11 - 15) deal with 31-day months, where the first five deal with months other than December, and the second five deal with December. There are no impossible rules in this portion of the decision table, though there is some redundancy that an efficient tester might question. Eight of the ten rules simply increment the day. Finally, the last seven rules focus on February and leap year.

Corresponding Test Cases:

Table 2.17: Third Try Decision table Test cases for NextDate function

Case ID	Month	Day	Year	Expected Output
1	April	15	1993	April 16, 1993
2	April	28	1993	April 29, 1993
3	April	29	1993	April 30, 1993

4	April	30	1993	May 1, 1993
5	April	31	1993	Impossible
6	Jan.	15	1993	Jan. 16, 1993
7	Jan.	28	1993	Jan. 29, 1993
8	Jan.	29	1993	Jan. 30, 1993
9	Jan.	30	1993	Jan. 31, 1993
10	Jan.	31	1993	Feb. 1, 1993
11	Dec.	15	1993	Dec. 16, 1993
12	Dec.	28	1993	Dec. 29, 1993
13	Dec.	29	1993	Dec. 30, 1993
14	Dec.	30	1993	Dec. 31, 1993
15	Dec.	31	1993	Jan. 1, 1994
16	Feb.	15	1993	Feb. 16, 1993
17	Feb.	28	1992	Feb. 29, 1992
18	Feb.	28	1993	Mar. 1, 1993
19	Feb.	29	1992	Mar. 1, 1992
20	Feb.	29	1993	Impossible
21	Feb.	30	1993	Impossible
22	Feb.	31	1993	Impossible

Test Cases for the Commission Problem

The functional view of the entire problem is that the salesperson's commission is a function of the telegrams:

$F(\text{telegrams}) = \text{commission}$

This decomposes naturally into three subfunctions:

$F_1(\text{telegrams}) = (\text{locks, stocks, barrels})$

$F_2(\text{locks, stocks, barrels}) = \text{sales}$

$F_3(\text{sales}) = \text{commission}$

The Commission Problem is just the composition of F_2 and F_3 . The input domain is partitioned by the limits on locks, stocks, and barrels.

$L_1 = \{\text{locks: } 1 \leq \text{locks} \leq 70\}$

$L_2 = \{\text{locks} = -1\}$ (occurs if locks = -1 is used to control input iteration)

$S_1 = \{\text{stocks}: 1 \leq \text{stocks} \leq 80\}$

$B_1 = \{\text{barrels}: 1 \leq \text{barrels} \leq 90\}$

The corresponding invalid classes of the input variables are:

$L_3 = \{\text{locks}: \text{locks} = 0 \text{ OR } \text{locks} < -1\}$

$L_4 = \{\text{locks}: \text{locks} > 70\}$

$S_2 = \{\text{stocks}: \text{stocks} < 1\}$

$S_3 = \{\text{stocks}: \text{stocks} > 80\}$

$B_2 = \{\text{barrels}: \text{barrels} < 1\}$

$B_3 = \{\text{barrels}: \text{barrels} > 90\}$

Table 2.18: Test Cases for the Commission Problem

Test Case	locks	stocks	barrels	sales	commission
DT1	5	5	5	500	50
DT2	15	15	15	1500	175
DT3	25	25	25	2500	360

The original problem statement addresses the cases where c_2 , c_5 , and c_8 are all true. There is no information for what should happen if any of the remaining conditions are true. This is an error of omission at specification time. We might end up with actions such as the following:

- a1: Error: must sell at least one rifle lock.
- a2: Error: cannot sell more than 70 rifle locks.
- a3: Error: must sell at least one rifle stock.
- a4: Error: cannot sell more than 80 rifle stocks.
- a5: Error: must sell at least one rifle barrel.
- a6: Error: cannot sell more than 90 rifle barrels.

Guidelines and Observations

As with the other testing techniques, decision table based testing works well for some applications (like NextDate) and is not worth the trouble for others (like Commission Problem). Not surprisingly, the situations in which it works well are those where there is a lot of decision making (like the Triangle Problem), and those in which there are important logical relationships among input variables (like the NextDate function).

1. The decision table technique is indicated for applications characterized by any of the following: prominent If-Then-Else logic logical relationships among input variables

calculations involving subsets of the input variables cause and effect relationships between inputs and outputs high cyclomatic (McCabe) complexity

2. Decision tables don't scale up very well (a limited entry table with n conditions has 2^n rules). There are several ways to deal with this: use extended entry decision tables, algebraically simplify tables, "factor" large tables into smaller ones, and look for repeating patterns of condition entries. For more on these techniques
3. As with other techniques, iteration helps. The first set of conditions and actions you identify may be unsatisfactory. Use it as a stepping stone, and gradually improve on it until you are satisfied with a decision table.

Fault-Based Testing

A model of potential program faults is a valuable source of information for evaluating and designing test suites. Some fault knowledge is commonly used in functional and structural testing, for example when identifying singleton and error values for parameter characteristics in category- partition testing or when populating catalogs with erroneous values, but a fault model can also be used more directly. Fault-based testing uses a fault model directly to hypothesize potential faults in a program under test, as well as to create or evaluate test suites based on its efficacy in detecting those hypothetical faults.

Q 9.) Explain fault-based testing with its terminologies and assumptions.

Q) Explain overview of assumptions in fault-based testing

Q) What is fault-based testing? Define below with respect to f fault-based testing:

i) Original program

ii) Program location

iii) Alternate Expression

iv) Alternate Program

Assumptions in Fault-Based Testing

The effectiveness of fault-based testing depends on the quality of the fault model and on some basic assumptions about the relation of the seeded faults to faults. The seeded faults are small syntactic changes, like replacing one variable reference by another in an expression, or changing a comparison from $<$ to $<=$.

If the program under test has an actual fault, we may hypothesize that it differs from another, corrected program by only a small textual change. If so, then we need merely distinguish the program from all such small variants to ensure detection of all such faults. This is known as the competent programmer hypothesis, an assumption that the program under test is "close to" a correct program.

Fault-based testing can guarantee fault detection only if the competent programmer hypothesis and the coupling effect hypothesis hold. This also implies that developing better fault models, based on hard data about real faults.

Fault-Based Testing: Terminology

Original program: The program unit (e.g., C function or Java class) to be tested.

Program location: A region in the source code. The precise definition is defined relative to the syntax of a particular programming language. Typical locations are statements, arithmetic and Boolean expressions, and procedure calls.

Alternate expression: Source code text that can be legally substituted for the text at a program location. A substitution is legal if the resulting program is syntactically correct (i.e., it compiles without errors).

Alternate program: A program obtained from the original program by substituting an alternate expression for the text at some program location.

Distinct behavior of an alternate program R for a test t: The behavior of an alternate program R is distinct from the behavior of the original program P for a test T I f R and P produce a different result for t, or if the output of R is not defined for t.

Distinguished set of alternate programs for a test suite T: A set of alternate programs are distinct if each alternate program in the set can be distinguished from the original program by at least one test in T.

Fault-based testing can guarantee fault detection only if the competent programmer hypothesis and the coupling effect hypothesis hold.

Fault-based testing techniques can be useful even if we decline to take the leap of faith required to fully accept their underlying assumptions.

Mutation Analysis

Q 10.) What is mutation? Explain variations on mutation in detail

Mutation analysis is the most common form of software fault-based testing. A fault model is used to produce hypothetical faulty programs by creating variants of the program under test. Variants are created by "seeding" faults, that is, by making a small change to the program under test following a pattern in the fault model. The patterns for changing program text are called **mutation operators**, and each variant program is called a **mutant**.

Mutation Analysis: Terminology

Original program under test: The program or procedure (function) to be tested.

Mutant A program differs the original program for one syntactic element (e.g., a statement, a condition, a variable, a label).

Distinguished mutant A mutant that can be distinguished for the original program by executing at least one test case.

Equivalent mutant A mutant that cannot be distinguished from the original program.

Mutation operator A rule for producing a mutant program by syntactically modifying the original program. Mutants should be plausible as faulty programs. Mutant programs that are rejected by a compiler, or that fail almost all tests, are not good models of the faults we seek to uncover with systematic testing.

Fault-Based Adequacy Criteria

Given a program and a test suite T , mutation analysis consists of the following steps:

Select mutation operators If we are interested in specific classes of faults, we may select a set of mutation operators relevant to those faults.

Generate mutants

Mutants are generated mechanically by applying mutation operators to the original program.

Distinguish mutants Execute the original program and each generated mutant with the test cases in T . A mutant is *killed* when it can be distinguished from the original program.

A mutant can remain *live* for two reasons:

- The mutant can be distinguished from the original program, but the test suite T does not contain a test case that distinguishes them (i.e., the test suite is not adequate with respect to the mutant).
- The mutant cannot be distinguished from the original program by any test case (i.e., the mutant is equivalent to the original program).

Given a set of mutants SM and a test suite T , the fraction of nonequivalent mutants killed by T measures the adequacy of T with respect to SM . Unfortunately, the problem of identifying equivalent mutants is undecidable either by claiming that a mutant is equivalent to the program under test when it is not or by counting some equivalent mutants among the remaining live mutants.

However, we can easily observe that mutant M_i is equivalent to the original program (i.e., no input would distinguish it). Conversely, mutants M_k and M_l seem to be nonequivalent to the original program: There should be at least one test case that distinguishes each of them from the original program. Thus, the adequacy of TS , measured after eliminating the equivalent mutant M_i , is 33%.

Variations on Mutation Analysis

The mutation analysis process kills mutants based on the outputs produced by execution of test cases, is known as **strong mutation**.

- It can generate a number of mutants quadratic in the size of the program.
- Each mutant must be compiled and executed with each test case until it is killed.
- The time and space required for compiling all mutants and for executing all test cases for each mutant may be impractical.
- The computational effort required for mutation analysis can be reduced by decreasing the number of mutants generated and the number of test cases to be executed.

Weak mutation analysis decreases the number of tests to be executed by killing mutants when they produce a different intermediate state, rather than waiting for a difference in the final result or observable program behavior. weak mutation analysis.

- With weak mutation, a single program can be seeded with many faults.
- A "meta mutant" program is divided into segments containing original and mutated source code, with a mechanism to select which segments to execute.
- Two copies of the meta-mutant are executed in tandem, one with only original program code selected and the other with a set of live mutants selected.
- Execution is paused after each segment to compare the program state of the two versions. If the state is equivalent, execution resumes with the next segment of original and mutated code. If the state differs, the mutant is marked as dead, and execution of original and mutated code is restarted with a new selection of live mutants.

Estimating Population Sizes: Counting fish

Lake Winnemunchie is inhabited by two kinds of fish, a native trout and an introduced species of chub. The Fish and Wildlife Service wishes to estimate the populations to evaluate their efforts to eradicate the chub without harming the population of native trout.

The population of chub can be estimated statistically as follows. 1000 chub are netted, their dorsal fins are marked by attaching a tag, then they are released back into the lake. Over the next weeks, fishermen are asked to report the number of tagged and untagged chub caught. If 50 tagged chub and 300 untagged chub are caught, we can calculate and thus there are about 6000 untagged chub remaining in the lake.

$$\frac{1000}{\text{untagged chub population}} = \frac{50}{300}$$

It may be tempting to also ask fishermen to report the number of trout caught and to perform a similar calculation to estimate the ratio between chub and trout. However, this is valid only if trout and chub are equally easy to catch, or if one can adjust the ratio using a known model of trout and chub vulnerability to fishing.

Counting residual faults

A similar procedure can be used to estimate the number of faults in a program: Seed a given number S of faults in the program. Test the program with some test suite and count the number

of revealed faults. Measure the number of seeded faults detected, DS , and also the number of natural faults DN detected. Estimate the total number of faults remaining in the program, assuming the test suite is as effective at finding natural faults as it is at finding seeded faults, using the formula

$$\frac{S}{\text{total natural faults}} = \frac{DS}{DN}$$

If we estimate the number of faults remaining in a program by determining the proportion of seeded faults detected, we must be wary of the pitfall of estimating trout population by counting chub. The seeded faults are chub, the real faults are trout, and we must either have good reason for believing the seeded faults are no easier to detect than real remaining faults, or else make adequate allowances for uncertainty. The difference is that we cannot avoid the problem by repeating the process with trout - once a fault has been detected, our knowledge of its presence cannot be erased. We depend, therefore, on a very good fault model, so that the chub are as representative as possible of trout.

Hardware Fault-based Testing

Fault-based testing is widely used for semiconductor and hardware system validation and evaluation both for evaluating the quality of test suites and for evaluating fault tolerance.

Semiconductor testing has conventionally been aimed at detecting random errors in fabrication, rather than design faults. Relatively simple fault models have been developed for testing semiconductor memory devices, the prototypical faults being "stuck-at-0" and "stuck-at-1" (a gate, cell, or pin that produces the same logical value regardless of inputs). A number of more complex fault models have been developed for particular kinds of semiconductor devices (e.g., failures of simultaneous access in dual port memories). A test vector (analogous to a test suite for software) can be judged by the number of hypothetical faults it can detect, as a fraction of all possible faults under the model.

In evaluation of fault tolerance in hardware, the usual approach is to modify the state or behavior rather than the system under test. Due a difference in terminology between hardware and software the corruption of state or modification of behavior is called a "fault," and artificially introducing it is called "fault injection." Pin-level fault injection consists of forcing a stuck-at-0, a stuck-at-1, or an intermediate voltage level (a level that is neither a logical 0 nor a logical 1) on a pin of a semiconductor device.

Heavy ion radiation is also used to inject random faults in a running system. A third approach, growing in importance as hardware complexity increases, uses software to modify the state of a running system or to simulate faults in a running simulation of hardware logic design. Fault seeding can be used statistically in another way: To estimate the number of faults remaining in a program.

VTU Question Paper Questions

Q No.	Question	Marks	Year
1.	Explain boundary value analysis and write the test cases using boundary value analysis for triangle problem	7	Dec-2019/Jan-2020
2.	Write a short note on decision table with an example	5	Dec-2019/Jan-2020
3.	Explain overview of assumptions in fault based testing	4	Dec-2019/Jan-2020
4.	Explain weak normal, weak robust and strong robust equivalence class testing with next date problem as an example	8	Dec-2019/Jan-2020
5.	What are the limitations of boundary value analysis?	4	Dec-2019/Jan-2020
6.	Explain variations on mutation analysis	4	Dec-2019/Jan-2020
7.	Explain boundary value analysis and BVA robust in detail with function of two variables and show how to prepare test input sets.	8	June/July-2019
8.	What is mutation? Explain variations on mutation in detail	8	June/July-2019
9.	Explain different types of equivalence class testing in detail	8	June/July-2019
10.	What is fault based testing? Define below with respect to f i)Original program ii) Program location iii) Alternate Expression iv) Alternate Program	8	June/July-2019
11.	Explain boundary value analysis. Mention its limitations. Derive BVA test cases for the triangle problem	8	June/July-2018
12.	Briefly explain the variants of equivalence class testing. Derive equivalence class test cases for next date problem	8	June/July-2018
13.	Explain the format of decision table. Build decision table for simple version of triangle problem.	8	June/July-2018
14.	Explain fault based testing with its terminologies and assumptions.	8	June/July-2018

Prepared by

Usha G.R.

Assistant Professor

Dept. of ISE